POSTGRES · INTERMEDIATE TO ADVANCED

Twenty Patterns Worth Learning, Read Off a Real Schema

Every snippet in this document is copied verbatim from the Hospital Commission Forms Platform's own migrations. Nothing here is a toy example: each one was written to solve a problem that a simpler approach had already failed to solve.

Source supabase/migrations/ · 475 migration files
Target PostgreSQL 15+ (this schema runs on 17) with pgcrypto
Compiled 2026-08-26

Contents

Roughly ordered: access control, then schema design, then concurrency and triggers.

01	RLS and GRANT are two independent gates — you need both	SECURITY
02	USING gates which rows you may touch; WITH CHECK gates the row you leave behind	SECURITY
03	Wrap volatile calls in a scalar subquery so the planner hoists them into an InitPlan	PERFORMANCE
04	SECURITY DEFINER without a pinned search_path is a privilege-escalation hole	SECURITY
05	EXECUTE on a new function is granted to PUBLIC by default — revoke it	SECURITY
06	SECURITY INVOKER + DISTINCT ON: push work into SQL without widening who can see what	PERFORMANCE
07	Model polymorphism as column-per-variant plus a discriminated CHECK, not a bare scope_id	SCHEMA DESIGN
08	A partial unique index is a conditional constraint	SCHEMA DESIGN
09	NULLS NOT DISTINCT makes NULL behave like a value inside a unique key	SCHEMA DESIGN
10	A composite foreign key can enforce cross-table tenant consistency	SCHEMA DESIGN
11	Generated columns turn a business rule into a foreign key (and NULL is how a row opts out)	SCHEMA DESIGN
12	DEFERRABLE constraints let a multi-row shuffle be transiently invalid	TRANSACTIONS
13	pg_advisory_xact_lock serialises a critical section without locking any row	CONCURRENCY
14	Hash chaining makes an append-only table tamper-evident	DATA INTEGRITY
15	HMAC with a server-side pepper gives you a searchable blind index over sensitive data	SECURITY
16	Statement-level triggers with transition tables replace N row-trigger firings with one	TRIGGERS
17	Row-level triggers never fire on TRUNCATE — guard it separately	TRIGGERS
18	Transaction-local GUCs are the clean handshake between an RPC and its triggers	TRANSACTIONS
19	SELECT … FOR UPDATE is the pessimistic lock for read-check-write inside an RPC	CONCURRENCY
20	unnest(...) WITH ORDINALITY turns an array parameter into a set — reorder in one statement	QUERY DESIGN

01 SECURITY · ROW-LEVEL SECURITY

RLS and GRANT are two independent gates — you need both

The `memberships` table is created with RLS on from the first statement, then given *only* `SELECT` to the `authenticated` role. There is no INSERT/UPDATE/DELETE policy and no DML grant, so a direct PostgREST write is refused at the privilege layer before any policy is even consulted. Writes must go through a SECURITY DEFINER door function.

supabase/migrations/20260720000000_memberships_table.sql:99-136

```
alter table public.memberships enable row level security;

-- Table-level privilege: SELECT only for `authenticated` (RLS gates the ROWS; the
-- GRANT gates the PRIVILEGE — both are needed for an RLS-scoped read to resolve).
grant select on public.memberships to authenticated;

create policy memberships_select on public.memberships
  for select to authenticated
  using (
    -- self-read: your own grants, every scope.
    principal_id = (select auth.uid())
    or app.is_admin()
    -- commission-scope rows.
    or (commission_id is not null and (
          app.is_member_of(commission_id) or app.is_commission_admin_of(commission_id)))
    -- organization-scope rows.
    or (organization_id is not null and commission_id is null and hospital_id is null
        and app.is_org_admin_of(organization_id))
    -- hospital-scope rows.
    or (hospital_id is not null and (
          app.is_org_admin_of(app.org_of_hospital(hospital_id))
          or app.is_hospital_admin_of(hospital_id)))
  );
```

TAKEAWAYS

- `ENABLE ROW LEVEL SECURITY` filters *rows*. `GRANT` decides whether the role may touch the table *at all*. A missing GRANT is a hard permission error; a missing policy silently returns zero rows.
- RLS with no policy for a command = deny-all for that command. That is a legitimate design: "no write policy" is how you force every write through an audited function.
- Always write `TO authenticated` (or another explicit role) on a policy. A policy with no `TO` clause applies to `PUBLIC`, which is wider than almost anyone intends.
- Table owners and superusers bypass RLS by default — that is why a service role can still seed the table. Use `ALTER TABLE ... FORCE ROW LEVEL SECURITY` if the owner should be subject to policies too.

02 SECURITY · ROW-LEVEL SECURITY

USING gates which rows you may touch; WITH CHECK gates the row you leave behind

Three policies on `responses` that split read, create and update. Note the asymmetry on UPDATE: `USING` requires the row be your own *and still a draft*, while `WITH CHECK` only requires the resulting row still be yours — so you may submit a draft (changing its status out of the predicate), but never edit somebody else's row and never resurrect a submitted one.

supabase/migrations/20260711000800_perf_indexes.sql:110-124

```
create policy responses_select on public.responses for select to authenticated
  using (created_by = (select auth.uid())
         or app.is_commission_admin_of(commission_id)
         or (status = 'submitted' and app.is_staff_admin_of(commission_id)));

create policy responses_insert_own on public.responses for insert to authenticated
  with check (created_by = (select auth.uid()) and app.is_member_of(commission_id));

create policy responses_update_own_draft on public.responses for update to authenticated
  using (created_by = (select auth.uid()) and status = 'in_progress')
  with check (created_by = (select auth.uid()));
```

TAKEAWAYS

- `USING` is evaluated against the row as it exists *now* (SELECT, UPDATE, DELETE). `WITH CHECK` is evaluated against the row as it will exist *after* the write (INSERT, UPDATE).
- INSERT has no `USING` clause — there is no pre-existing row. DELETE has no `WITH CHECK` — there is no resulting row.
- If you write only `USING` on an UPDATE policy, Postgres reuses it as the `WITH CHECK`. Often convenient, but it silently forbids the row from leaving the predicate — here it would have blocked `status` ever moving off `'in_progress'`.
- A classic real-world bug: putting a state gate only in `WITH CHECK`. It constrains the new value but never restricts *which* rows you were allowed to modify.

03 PERFORMANCE · RLS

Wrap volatile calls in a scalar subquery so the planner hoists them into an InitPlan

Every policy in this codebase writes `(select auth.uid())` rather than a bare `auth.uid()`. Semantically identical; operationally very different — the bare call is re-evaluated for every candidate row, the wrapped one is evaluated once per statement and cached as an InitPlan.

supabase/migrations/20260711000800_perf_indexes.sql:132-140

```
create policy organization_members_select on public.organization_members
  for select to authenticated
  using (app.is_admin()
         or app.is_org_admin_of(organization_id)
         or user_id = (select auth.uid()));

comment on policy organization_members_select on public.organization_members is
  'A member reads their OWN grants (user_id = auth.uid()) so getSessionContext can '
  'resolve hospital_admin/nsp_org_admin standing; an admin/org_admin reads the full '
  'org roster. Minimum-necessary. WS-5 P9: auth.uid() InitPlan-wrapped '
  '(semantically identical).';
```

TAKEAWAYS

- A policy predicate runs once per row scanned. On a 100k-row table an unwrapped `auth.uid()` is 100k function calls.
- `(select f())` is an *uncorrelated* subquery — it references no outer column — so the planner evaluates it once and substitutes the result. This is a general Postgres technique, not a Supabase one.
- It also affects index usage: `user_id = (select auth.uid())` can drive an index scan against a constant, where a per-row volatile call often degrades into a sequential scan.
- Verify rather than assume: `EXPLAIN (ANALYZE, BUFFERS)` run as the target role shows an `InitPlan 1` node when the hoist happened.
- Mark your own helper functions `STABLE` wherever truthful. `VOLATILE` is the default and forbids most of these optimisations.

04 SECURITY · FUNCTIONS

SECURITY DEFINER without a pinned search_path is a privilege-escalation hole

A DEFINER function runs with the *owner's* rights and bypasses RLS entirely. Every DEFINER function here pins `search_path` and schema-qualifies its references, so a caller cannot create a shadow `app_secrets` in a schema they control and have the function resolve to it. Note also that the body re-implements its own authorization checks, because RLS is no longer protecting it.

supabase/migrations/20260620000000_baseline.sql:1834-1856

```
CREATE OR REPLACE FUNCTION "app"."derive_patient_key"("p_value" "text") RETURNS "text"
    LANGUAGE "plpgsql" STABLE SECURITY DEFINER
    SET "search_path" TO 'app', 'pg_catalog'
    AS $$
declare
  v_pepper text;
  v_norm   text;
begin
  select nullif(btrim(value), '') into v_pepper
  from app.app_secrets where key = 'mrn_pepper';

  if v_pepper is null then
    raise exception 'o segredo app_secrets[''mrn_pepper''] não está configurado'
      using errcode = 'check_violation';
  end if;

  v_norm := app.normalize_identifier(p_value);
  if v_norm is null then
    return null;
  end if;
  return encode(extensions.hmac(v_norm, v_pepper, 'sha256'), 'hex');
end;
$$;

ALTER FUNCTION "app"."derive_patient_key"("p_value" "text") OWNER TO "postgres";
```

TAKEAWAYS

- `SET search_path` on a function is a per-call setting the caller cannot override. Without it, anyone who can create objects in a schema on the caller's search_path can hijack an unqualified reference and have it run as the owner.
- Pin it to the minimum: the schemas the body genuinely uses, plus `pg_catalog`. The strictest form is `SET search_path = ''` with every name fully qualified.
- The function's *owner* is the identity it runs as, so `ALTER FUNCTION ... OWNER TO postgres` is a load-bearing statement, not boilerplate.
- A DEFINER function's internal `IF NOT ...` gate *replaces* RLS — it does not add to it. Reviewing `pg_policies` alone tells you nothing about what a DEFINER function exposes; read `pg_proc.prosecdef` alongside it.
- Prefer `SECURITY INVOKER` (the default) unless you specifically need to cross a privilege boundary.

05 SECURITY · PRIVILEGES

EXECUTE on a new function is granted to PUBLIC by default — revoke it

Two related hardening moves. Per object: every door function revokes from `PUBLIC` first, then grants to named roles. Per schema: the stock blanket `ALTER DEFAULT PRIVILEGES ... GRANT ALL TO authenticated` is flipped to revoke-by-default, so a table created later without RLS is not automatically world-readable.

supabase/migrations/20260709000200_commission_admin_predicate.sql:110-117 · 20260711000100_grant_hardening.sql:30, 91-93

```
-- Per-function: close the PUBLIC default, then open to exactly two roles.
revoke all on function app.is_hospital_admin_of(uuid)   from public;
revoke all on function app.is_commission_admin_of(uuid) from public;
grant execute on function app.is_hospital_admin_of(uuid)   to authenticated, service_role;
grant execute on function app.is_commission_admin_of(uuid) to authenticated, service_role;

-- Table-level: the tamper-evidence table takes no client DML at all.
revoke insert, update, delete, truncate on public.audit_log from authenticated;

-- Posture flip: stop handing every FUTURE object to authenticated automatically.
alter default privileges for role postgres in schema public
  revoke all on tables    from authenticated;
alter default privileges for role postgres in schema public
  revoke all on functions from authenticated;
alter default privileges for role postgres in schema public
  revoke all on sequences from authenticated;
```

TAKEAWAYS

- Postgres grants `EXECUTE` on every new function to `PUBLIC`, which means every role — including roles created later. A `NULL proacl` in `pg_proc` means "defaults apply", i.e. PUBLIC can execute.
- The safe idiom is always the pair: `REVOKE ALL ... FROM PUBLIC` then `GRANT EXECUTE ... TO <named roles>`. Granting without revoking leaves the default wide open.
- `ALTER DEFAULT PRIVILEGES` is scoped to a specific *role and schema* and only affects objects created *after* it runs. It never retroactively changes existing objects.
- A `REVOKE` you are not entitled to issue is a silent no-op, not an error. Verify with `\dp`, `relacl` and `proacl` rather than trusting that the statement had an effect.

06 PERFORMANCE · QUERY DESIGN

SECURITY INVOKER + DISTINCT ON: push work into SQL without widening who can see what

This replaced a client-side full scan that fetched every audit row just to build a filter dropdown. Declaring the function `SECURITY INVOKER` means the caller's RLS on `audit_log` is still the authority, so pushing the work into the database did not change visibility. `DISTINCT ON` then collapses to one row per actor in a single pass.

supabase/migrations/20260711000900_perf_sweep_wave2.sql:36-48

```
create or replace function public.list_audit_filter_actors(p_commission uuid default null)
returns table (actor_id uuid, full_name text)
language sql
stable
security invoker
set search_path to 'public', 'pg_catalog'
as $$
  select distinct on (al.actor_id) al.actor_id, p.full_name
  from public.audit_log al
  left join public.profiles p on p.id = al.actor_id
  where p_commission is null or al.commission_id = p_commission
  order by al.actor_id, p.full_name;
$$;
```

TAKEAWAYS

- `DISTINCT ON (expr)` is a Postgres extension that keeps the *first* row per group. The `ORDER BY` must begin with the same expressions as the `DISTINCT ON` list; the remaining ORDER BY terms decide *which* row wins.
- It is usually cheaper than the equivalent `row_number() OVER (PARTITION BY ...)` plus an outer filter, and far cheaper than `GROUP BY` when you want whole rows rather than aggregates.
- `SECURITY INVOKER` is the default and the correct choice whenever RLS should remain the authority. Reach for DEFINER only to deliberately exceed the caller's rights.
- A `LANGUAGE sql` function (rather than `plpgsql`) that is simple and `STABLE` can be *inlined* by the planner, merging its predicate into the calling query — plpgsql is always an opaque call boundary.

07 SCHEMA DESIGN · CONSTRAINTS

Model polymorphism as column-per-variant plus a discriminated CHECK, not a bare scope_id

One membership table serves organization-, hospital- and commission-scoped roles. Instead of an untyped `scope_id uuid` (which can carry no foreign key), each scope gets a real FK column, and a `CASE`-based CHECK asserts exactly which columns each role may populate. The `ELSE false` arm makes an unknown role structurally impossible.

supabase/migrations/20260720000000_memberships_table.sql:42-71

```
constraint memberships_role_check check (role = any (array[
  'org_admin', 'nsp_org_admin', 'hospital_admin', 'nsp_coordinator',
  'staff_admin', 'staff', 'pqs_member'
])),

-- DISCRIMINATED SHAPE CHECK — exactly the scope columns each role carries.
constraint memberships_scope_shape check (
  case role
    when 'org_admin'       then organization_id is not null and hospital_id is null     and commission_id is null
    when 'nsp_org_admin'   then organization_id is not null and hospital_id is null     and commission_id is null
    when 'hospital_admin'  then organization_id is not null and hospital_id is not null and commission_id is null
    when 'nsp_coordinator' then organization_id is not null and hospital_id is not null and commission_id is null
    when 'pqs_member'      then organization_id is not null and hospital_id is not null and commission_id is null
    when 'staff_admin'     then commission_id   is not null and organization_id is null and hospital_id is null
    when 'staff'           then commission_id   is not null and organization_id is null and hospital_id is null
    else false   -- unknown role rejected (belt with the role CHECK above)
  end
),

-- title_id legal ONLY on commission-scope rows.
constraint memberships_title_scope check (title_id is null or commission_id is not null)
```

TAKEAWAYS

- A nullable FK column per variant preserves referential integrity, `ON DELETE CASCADE` and index-ability. A generic `scope_id` throws away all three.
- The `ELSE false` is the important half: it turns the constraint from a blocklist into an allowlist, so adding a role to the enum without updating the shape rule fails loudly instead of silently permitting anything.
- A CHECK constraint may only reference columns of its own row. Anything cross-row or cross-table needs a trigger, an exclusion constraint, or a foreign key.
- CHECK constraints treat `NULL` as passing (unknown is not false). Write predicates so the NULL case is handled explicitly, as the `is null` / `is not null` tests do here.
- Worth knowing as a one-liner alternative: `num_nonnulls(a, b, c) = 1` when you only need "exactly one of these is set".

08 SCHEMA DESIGN · INDEXES

A partial unique index is a conditional constraint

Four real uses from this schema: at most one published version per form; uniqueness of an answer that applies only at the top level; a per-tenant monotonic audit sequence; and per-scope-tier indexes sized to hold only the rows of that tier.

supabase/migrations/20260620000000_baseline.sql:19279, 19295, 19563 · 20260720000000_memberships_table.sql:90-95

```
-- At most ONE published version per form; unlimited drafts and archived rows.
create unique index form_versions_one_published_idx
  on public.form_versions (form_id) where (status = 'published');

-- Uniqueness that applies only to top-level answers.
create unique index answers_uq_top
  on public.answers (response_id, item_id) where (group_instance_id is null);

-- A gap-free sequence per commission chain.
create unique index audit_log_commission_seq_key
  on public.audit_log (commission_id, seq) where (commission_id is not null);

-- Partial so each index covers exactly one scope tier — smaller, hotter, cheaper.
create index memberships_commission_idx on public.memberships (commission_id, principal_id)
  where commission_id is not null;
create index memberships_hospital_idx   on public.memberships (hospital_id, principal_id, role)
  where hospital_id is not null;
```

TAKEAWAYS

- `CREATE UNIQUE INDEX ... WHERE <pred>` expresses rules a plain `UNIQUE` constraint cannot: "at most one row may be in state X".
- There is no `ALTER TABLE ... ADD CONSTRAINT ... WHERE`. Partial uniqueness must be an index — which also means it cannot be the target of a foreign key.
- The planner uses a partial index only when it can prove the query's `WHERE` implies the index predicate. Keep predicates simple and mirror them in your queries.
- `ON CONFLICT` can target a partial index through its inference clause: `on conflict (response_id, item_id) where group_instance_id is null`.
- A partial index on a soft-delete column (`where deleted_at is null`) is one of the highest-value, lowest-effort wins in most schemas.

09 SCHEMA DESIGN · CONSTRAINTS

NULLS NOT DISTINCT makes NULL behave like a value inside a unique key

The rule is "one grant per (principal, exact scope, role)", but two of the three scope columns are NULL on any given row. Under standard SQL semantics NULL is never equal to NULL, so the default index would happily accept unlimited duplicates. `NULLS NOT DISTINCT` (Postgres 15+) collapses them into one logical key.

supabase/migrations/20260720000000_memberships_table.sql:81-86

```sql
-- One grant per (principal, exact scope, role). NULLS NOT DISTINCT (PG15+) so the
-- NULL scope columns collapse to one logical key.
create unique index memberships_grant_uq on public.memberships
  using btree (principal_id, role, organization_id, hospital_id, commission_id)
  nulls not distinct;
```

TAKEAWAYS

- By default a unique index treats every NULL as distinct, so `(u1, 'staff', NULL, NULL, c1)` could be inserted a thousand times. This is a common and very quiet data-quality bug.
- `NULLS NOT DISTINCT` (PG 15+) is the direct fix. Before PG 15 the workarounds were one partial unique index per shape, or a `COALESCE(col, <sentinel>)` expression index.
- It works on table constraints too: `unique nulls not distinct (a, b)`.
- Remember the sibling asymmetry: `where x = NULL` is never true. Use `IS NULL`, or `IS DISTINCT FROM` for NULL-safe inequality.

10 SCHEMA DESIGN · INTEGRITY

A composite foreign key can enforce cross-table tenant consistency

`commissions` carried two independent FKs — `hospital_id` and `organization_id` — with nothing tying the stored org to the hospital's *actual* org. Repointing a hospital silently desynced every child row and every org-scoped RLS predicate. The fix adds an FK-referenceable superset `UNIQUE(id, organization_id)` on the parent, then a composite FK from the child, so the two columns must agree by construction.

supabase/migrations/20260711000400_tenant_composite_fk.sql:28-45, 57-80

```
-- 1 · FK-referenceable unique on the PARENT (trivially satisfied — id is the PK).
alter table public.hospitals
  add constraint hospitals_id_org_uq unique (id, organization_id);

-- 2 · Composite FK on the CHILD: the stored org must match the hospital's org.
--     KEEP the existing single-column FKs (their ON DELETE RESTRICT stays);
--     this adds the cross-tenant consistency check.
alter table public.commissions
  add constraint commissions_hospital_org_fkey
  foreign key (hospital_id, organization_id)
  references public.hospitals (id, organization_id);

-- 3 · A BEFORE UPDATE guard purely for the friendly error. The composite FK already
--     blocks the populated case (ON UPDATE NO ACTION), but opaquely.
create or replace function app.guard_hospital_org_repoint() returns trigger
  language plpgsql
  set search_path to 'app', 'public', 'pg_catalog'
as $$
begin
  if new.organization_id is distinct from old.organization_id
     and exists (select 1 from public.commissions where hospital_id = old.id) then
    raise exception 'não é permitido mover um hospital com comissões para outra organização'
      using errcode = 'HC082';
  end if;
  return new;
end;
$$;
```

TAKEAWAYS

- A foreign key may reference *any* unique constraint, not only the primary key. Adding a redundant `UNIQUE(pk, denormalized_col)` is the standard trick that makes a denormalized column FK-checkable.
- This is how you make a denormalized tenant id *safe*. Denormalizing for RLS performance is fine; unvalidated denormalization is a silent cross-tenant leak waiting to happen.
- Composite and single-column FKs coexist happily: keep the single-column one for its `ON DELETE` behaviour, and let the composite supply the cross-check.
- `IS DISTINCT FROM` is the NULL-safe `<>` — essential in triggers, where `new.col <> old.col` evaluates to NULL (and so skips the branch) whenever either side is NULL.
- Constraints give correctness; triggers give good error messages. Using both is layering, not redundancy.

11 SCHEMA DESIGN · ADVANCED CONSTRAINTS

Generated columns turn a business rule into a foreign key (and NULL is how a row opts out)

The rule is "a form item's parent must itself be a container, in the same form version, and nesting is capped at depth 1". Two `GENERATED ALWAYS AS ... STORED` columns give the FK something to reference. The trick is `parent_is_container`: it is deliberately NULL for a top-level item, and because a MATCH SIMPLE composite FK is not enforced when *any* of its columns is NULL, a top-level item opts out of the check automatically.

supabase/migrations/20260828000000_ff1_repeating_groups_schema.sql:120-152

```
-- (i) Two generated discriminators. Both expressions are IMMUTABLE.
alter table public.form_items
  add column is_container boolean
    generated always as (item_type = any (array['group','repeating_group'])) stored;

alter table public.form_items
  add column parent_is_container boolean
    generated always as (case when parent_item_id is null then null else true end) stored;

-- (ii) The referenced key for the composite parent FK.
alter table public.form_items
  add constraint form_items_container_uq unique (id, form_version_id, is_container);

-- (iii) Re-cut the self-FK. ON UPDATE NO ACTION (the default) is deliberate: it
--       blocks flipping a container's item_type while it still has children.
alter table public.form_items
  drop constraint form_items_parent_item_id_fkey;

alter table public.form_items
  add constraint form_items_parent_item_id_fkey
  foreign key (parent_item_id, form_version_id, parent_is_container)
  references public.form_items (id, form_version_id, is_container);
```

TAKEAWAYS

- A `GENERATED ALWAYS AS (...) STORED` column is computed on write and physically stored. The expression must be `IMMUTABLE` and may reference only columns of the same row — no subqueries, no `now()`.
- Because it is a real column, it can be indexed, made part of a `UNIQUE` constraint, and referenced by a foreign key. That is what lets you promote a predicate into declarative integrity.
- `MATCH SIMPLE` is the FK default: if any referencing column is NULL, the constraint is *not checked at all*. `MATCH FULL` instead demands all-NULL or none-NULL. Knowing which you have is the difference between a working opt-out and a silent hole.
- A self-referencing composite FK is a compact way to express "parents must be of kind X" without a trigger, and unlike a trigger it is enforced during bulk loads and by every write path.
- Postgres 18 adds `GENERATED ... VIRTUAL` (computed on read). Until then, `STORED` is the only option and it costs table width.

12 TRANSACTIONS · CONSTRAINTS

DEFERRABLE constraints let a multi-row shuffle be transiently invalid

Re-packing positions to a contiguous 0..n-1 range transiently collides on a `UNIQUE(parent, position)`, even when the final state is valid. The constraint is declared `DEFERRABLE`, and the RPC defers it for the duration of the shuffle — then forces the check back to `IMMEDIATE` *inside* the function, so a genuine violation surfaces as this function's error rather than as a bare 23505 at COMMIT in some unrelated statement's name.

supabase/migrations/20260709000100_commission_titles.sql:37-39 · 20260828000200_ff1_group_instance_rpcs.sql:208-227

```sql
-- DDL: DEFERRABLE so a reorder can shuffle positions in one txn.
alter table only public.commission_member_titles
  add constraint commission_member_titles_commission_position_key
  unique (commission_id, position) deferrable;

-- RPC: defer around the shuffle, then re-arm the check while still in scope.
set constraints public.response_group_instances_parent_position_uniq deferred;

update public.response_group_instances gi
set position = packed.new_position
from (
  select id, (row_number() over (order by position) - 1)::integer as new_position
  from public.response_group_instances
  where response_id = p_response_id
    and group_item_id = v_group_item_id
    and parent_instance_id is null
) packed
where gi.id = packed.id
  and gi.position <> packed.new_position;

set constraints public.response_group_instances_parent_position_uniq immediate;
```

TAKEAWAYS

- `DEFERRABLE INITIALLY IMMEDIATE` (the default when you write just `DEFERRABLE`) checks at statement end but *can* be deferred on demand. `DEFERRABLE INITIALLY DEFERRED` always waits for COMMIT.
- `SET CONSTRAINTS ... DEFERRED` lasts for the transaction. Flipping back to `IMMEDIATE` forces the check to run right there — the way to keep a good error message and a precise failure point.
- Only `UNIQUE`, `PRIMARY KEY`, `EXCLUDE` and `REFERENCES` constraints can be deferred. `CHECK` and `NOT NULL` cannot.
- A deferrable unique constraint cannot be used for `ON CONFLICT` inference — a real trade-off, and the reason this schema keeps some position uniques non-deferrable.
- The alternative when you cannot defer: a two-pass update that first offsets every row into a disjoint range (this codebase uses `set position = -position`) and then renumbers.

13 CONCURRENCY

pg_advisory_xact_lock serialises a critical section without locking any row

The audit log carries a gap-free monotonic `seq` per tenant chain. A sequence object cannot do this (sequences leak gaps on rollback), and `MAX(seq)+1` races. So the writer takes a transaction-scoped advisory lock keyed on a hash of the chain identity, reads the tail, and inserts — serialising writers to the *same* chain while leaving different chains fully concurrent.

supabase/migrations/20260620000000_baseline.sql:1045-1078

```
-- The CHAIN is identified by PRECEDENCE, not the raw (org, commission) tuple.
-- The lock + tail use the SAME chain identity so seq never collides.
if p_commission is not null then
  v_lock_key := 'audit:c:' || p_commission::text;
  perform pg_advisory_xact_lock(hashtextextended(v_lock_key, 0));
  select seq, row_hash into v_seq, v_prev_hash
  from public.audit_log
  where commission_id = p_commission
  order by seq desc
  limit 1;
elsif v_org is not null then
  v_lock_key := 'audit:o:' || v_org::text;
  perform pg_advisory_xact_lock(hashtextextended(v_lock_key, 0));
  select seq, row_hash into v_seq, v_prev_hash
  from public.audit_log
  where organization_id = v_org and commission_id is null
  order by seq desc
  limit 1;
else
  v_lock_key := 'audit:p';
  perform pg_advisory_xact_lock(hashtextextended(v_lock_key, 0));
  ...
end if;

v_seq := coalesce(v_seq, 0) + 1;
```

TAKEAWAYS

- Advisory locks are application-defined locks on an arbitrary 64-bit key. Nothing in the database enforces what the key means — the discipline is entirely yours.
- Use the `_xact_` variants (`pg_advisory_xact_lock`) so the lock is released automatically at COMMIT or ROLLBACK. The session-scoped variants must be released by hand and leak on any error path.
- `hashtextextended(text, seed)` turns a readable key like `'audit:c:<uuid>'` into the required bigint. Namespace your keys with a prefix so unrelated subsystems cannot collide.
- The lock and the read must use the *same* key derivation, or you have serialised nothing. Here that identity is stated in a comment precisely because it is the invariant.
- Sequences (`nextval`) are non-transactional by design — fast, but gappy. When gap-free numbering is a requirement, you are choosing to pay for serialisation.
- Related tool: `pg_try_advisory_xact_lock` returns false instead of waiting — good for "skip if someone else is already doing this" jobs.

14 DATA INTEGRITY · CRYPTOGRAPHY

Hash chaining makes an append-only table tamper-evident

Each audit row stores `prev_hash` (the previous row's hash) and `row_hash` = SHA-256 of `prev_hash` concatenated with a canonical serialisation of this row's fields. Editing or deleting any historical row breaks every hash after it. A separate verifier walks each chain in `seq` order and recomputes, returning the first broken sequence number.

supabase/migrations/20260620000000_baseline.sql:1079-1102, 17786-17806

```
-- WRITE: link this row to the previous one.
v_row_hash := encode(
  extensions.digest(
    coalesce(v_prev_hash, '') || app.audit_canonical(
      v_seq, v_occurred, v_actor, v_actor_is_admin, p_commission,
      p_action, p_entity_type, p_entity_id, p_summary,
      coalesce(p_metadata, '{}'::jsonb), v_org
    ),
    'sha256'
  ),
  'hex'
);

-- VERIFY: recompute the whole chain and report the first divergence.
v_expected := encode(
  extensions.digest(
    coalesce(v_prev_hash, '') || app.audit_canonical(
      v_rec.seq, v_rec.occurred_at, v_rec.actor_id, v_rec.actor_is_admin,
      v_rec.commission_id, v_rec.action, v_rec.entity_type, v_rec.entity_id,
      v_rec.summary, v_rec.metadata, v_rec.organization_id
    ),
    'sha256'
  ),
  'hex'
);
if v_rec.prev_hash is distinct from v_prev_hash or v_rec.row_hash <> v_expected then
  ok := false;
  broken_seq := v_rec.seq;
  return next;
  return;
end if;
v_prev_hash := v_rec.row_hash;
```

TAKEAWAYS

- `digest()` and `hmac()` come from the `pgcrypto` extension. Install it into its own schema (Supabase uses `extensions`) and reference it fully qualified.
- The *canonical serialisation* is the load-bearing part. If field order, NULL rendering, or timestamp formatting can vary, the verifier produces false alarms. Isolating it in one function (`audit_canonical`) is what keeps writer and verifier in agreement.
- Tamper-*evident* is not tamper-*proof*. Someone with write access can rewrite the whole chain. The property you gain is detection, which is why the verifier must run and be watched.
- `coalesce(v_prev_hash, '')` handles the genesis row. Every chain needs an explicit, documented starting condition.
- Note the interaction with partitioning: range-partitioning this table by time would force the partition key into the per-chain unique index and break the gap-free `seq` invariant the chain depends on. Integrity constraints and physical layout are not independent decisions.

15 SECURITY · DATA PROTECTION

HMAC with a server-side pepper gives you a searchable blind index over sensitive data

Patient identifiers must be matchable across records without storing the raw value in every table. `derive_patient_key` normalises the input, then returns `hmac(value, pepper, 'sha256')`. The result is deterministic — equal inputs give equal keys, so you can index and join on it — but not reversible without the pepper, which lives in a table only the function's owner can read.

supabase/migrations/20260620000000_baseline.sql:1834-1856

```
select nullif(btrim(value), '') into v_pepper
from app.app_secrets where key = 'mrn_pepper';

if v_pepper is null then
  raise exception 'o segredo app_secrets[''mrn_pepper''] não está configurado'
    using errcode = 'check_violation';
end if;

v_norm := app.normalize_identifier(p_value);
if v_norm is null then
  return null;
end if;

return encode(extensions.hmac(v_norm, v_pepper, 'sha256'), 'hex');

-- The derived key is what gets indexed on the cross-reference table:
--   create index patient_xref_patient_key_idx on public.patient_xref (patient_key)
--     where patient_key is not null;
```

TAKEAWAYS

- Use `hmac(data, key, type)`, not bare `digest()`, when the input space is small or guessable. A plain SHA-256 of a medical record number is trivially brute-forced; HMAC with a secret pepper is not.
- *Normalise before hashing.* Trim, case-fold, strip punctuation — whatever your equality rule is. If normalisation drifts, previously-matching records silently stop matching, and no constraint can tell you.
- Deterministic hashing is what makes the value indexable and joinable. That is the whole point, and also the trade-off: it leaks equality (you can tell two rows refer to the same person).
- Failing loudly when the pepper is missing matters. The dangerous alternative is a function that quietly returns a hash of an empty key and collapses every patient into one bucket.
- This is a pepper (one global secret, not stored with the data), not a salt (per-row, stored alongside). Per-row salts would break equality matching, which is why they are wrong here.

16 TRIGGERS · PERFORMANCE

Statement-level triggers with transition tables replace N row-trigger firings with one

Any change to an answer must bump a revision counter on the owning case. A row-level trigger would fire once per answer — hundreds of times for a bulk save. `REFERENCING NEW TABLE AS new_answers` exposes the entire set of affected rows to a single `FOR EACH STATEMENT` invocation, so the work becomes one set-based statement with a `GROUP BY`.

supabase/migrations/20261003002200_case_print_revision_substrate.sql:170-200, 362-375

```
create or replace function app.trg_bump_case_revision_answers_new()
returns trigger
language plpgsql
security definer
set search_path to 'app', 'public', 'pg_catalog'
as $$
begin
  perform app.bump_case_print_revision(cp.case_id)
  from new_answers a
  join public.responses r  on r.id  = a.response_id
  join public.case_phases cp on cp.id = r.case_phase_id
  group by cp.case_id;
  return null;
end;
$$;

create trigger bump_case_print_revision_ins after insert on public.answers
  referencing new table as new_answers
  for each statement execute function app.trg_bump_case_revision_answers_new();

create trigger bump_case_print_revision_upd after update on public.answers
  referencing new table as new_answers
  for each statement execute function app.trg_bump_case_revision_answers_new();

create trigger bump_case_print_revision_del after delete on public.answers
  referencing old table as old_answers
  for each statement execute function app.trg_bump_case_revision_answers_old();
```

TAKEAWAYS

- Transition tables (`REFERENCING NEW TABLE` / `OLD TABLE`) are only available on `AFTER FOR EACH STATEMENT` triggers. They behave like real relations you can join and aggregate.
- DELETE has no `NEW TABLE` and INSERT has no `OLD TABLE` — which is exactly why this has two functions and three triggers rather than one `INSERT OR UPDATE OR DELETE` trigger.
- A statement-level trigger fires once even when the statement affects *zero* rows. Row triggers do not fire at all. Design accordingly.
- A `FOR EACH STATEMENT` trigger has no `NEW` / `OLD` record and its return value is ignored — `return null` is the convention.
- General rule: reach for row-level triggers when you must veto or rewrite individual rows (`BEFORE`), and statement-level plus transition tables when you are propagating an effect.

17 TRIGGERS · DATA INTEGRITY

Row-level triggers never fire on TRUNCATE — guard it separately

The append-only guard on `audit_log` was a `BEFORE DELETE OR UPDATE FOR EACH ROW` trigger, which TRUNCATE bypasses entirely — including a TRUNCATE that arrives by CASCADE from a parent table. This adds a statement-level `BEFORE TRUNCATE` arm, with one explicit, GUC-gated escape hatch for deliberate maintenance.

supabase/migrations/20260711000100_grant_hardening.sql:47-63

```
create or replace function app.guard_audit_truncate() returns trigger
  language plpgsql
  set search_path to 'app', 'pg_catalog'
as $$
begin
  if coalesce(current_setting('app.allow_audit_teardown', true), 'off') = 'on' then
    return null;
  end if;
  raise exception 'os registros de auditoria são imutáveis (TRUNCATE bloqueado)'
    using errcode = 'HC042';
end;
$$;

create trigger guard_audit_immutable_truncate_trg
  before truncate on public.audit_log
  for each statement execute function app.guard_audit_truncate();
```

TAKEAWAYS

- TRUNCATE is DDL-flavoured DML: no row triggers, no RLS enforcement, no per-row `DELETE` visibility. Only a `BEFORE/AFTER TRUNCATE ... FOR EACH STATEMENT` trigger sees it.
- `TRUNCATE ... CASCADE` on a parent silently truncates every FK-referencing table. A guard on the parent alone is not a guard on the children.
- TRUNCATE is a separate privilege. Revoking it (`revoke truncate on ... from authenticated`) is the primary defence; the trigger is defence in depth behind it.
- `current_setting('name', true)` — note the second argument — returns NULL instead of raising when the setting is unset. Without it, an unset GUC raises and your guard fails in the wrong direction.
- An escape hatch should be explicit, named, and useless to an unprivileged caller. Here it is, because `authenticated` has no TRUNCATE grant no matter what the GUC says.

18 TRANSACTIONS · TRIGGERS

Transaction-local GUCs are the clean handshake between an RPC and its triggers

A published form version is immutable, but *the publish function itself* must be able to change its status. Rather than weakening the trigger, the RPC sets a transaction-local setting that the trigger reads. Any other path — a direct UPDATE, another function, a client — never sets it, so the immutability guard holds for everyone else.

supabase/migrations/20260620000000_baseline.sql:11358-11362, 12503-12515 · 20260720000910_referral_dialogue_rpcs.sql:75-84

```
-- In the RPC: third argument true = transaction-local (reverts at COMMIT/ROLLBACK).
perform set_config('app.in_publish_rpc', 'on', true);
--   ... perform the privileged status transition ...
perform set_config('app.in_publish_rpc', 'off', true);

-- In the trigger: read it NULL-safely and default to closed.
begin
  if tg_op = 'DELETE' then
    if old.status = 'published' then
      raise exception 'published versions are immutable (delete blocked)'
        using errcode = 'check_violation';
    end if;
    return old;
  end if;

  -- UPDATE. Status transitions are only permitted inside publish_form_version.
  if new.status is distinct from old.status then
    if coalesce(current_setting('app.in_publish_rpc', true), 'off') <> 'on' then
      raise exception 'version status changes must go through publish_form_version()'
        using errcode = 'check_violation';
    end if;
    return new;
  end if;

  -- Non-status update: forbidden once the version is no longer a draft.
  if old.status <> 'draft' then
    raise exception 'published/archived versions are immutable (update blocked)'
      using errcode = 'check_violation';
  end if;

  return new;
end;
```

TAKEAWAYS

- `set_config(name, value, is_local)` with `is_local = true` is the function-callable equivalent of `SET LOCAL` — it reverts at end of transaction, so it cannot leak into a later request on a pooled connection.
- Always read with `current_setting(name, true)` and `coalesce` to a safe default. The fail-closed default (`'off'`) is what makes an unset GUC deny rather than allow.
- Custom GUC names must contain a dot (`app.in_publish_rpc`). Namespace them by subsystem.
- This is a capability token, not a security boundary: it works because *only* the privileged function can reach the code that sets it. If a client could run arbitrary SQL, it could set the GUC too.
- The related trap: a bare `SET LOCAL` in a migration that is not running inside an explicit transaction is a *silent no-op* — Postgres emits warning 25P01 and carries on. Wrap it in a `DO $$ ... $$` block or an explicit BEGIN.

19 CONCURRENCY

SELECT ... FOR UPDATE is the pessimistic lock for read-check-write inside an RPC

Posting a message to a referral must validate the referral's current status and then append. Between the read and the write, another transaction could change that status. Loading the row `FOR UPDATE` takes a row-level exclusive lock held to the end of the transaction, so concurrent callers queue rather than both passing the same stale check.

supabase/migrations/20260720000910_referral_dialogue_rpcs.sql:42-73

```
select * into v_ref from public.case_referral where id = p_referral_id for update;
if v_ref.id is null then
  raise exception 'encaminhamento não encontrado' using errcode = 'no_data_found';
end if;

if not app.can_read_referral_phi(p_referral_id, v_uid) then
  raise exception 'você não pode enviar mensagens neste encaminhamento' using errcode = 'HC0A0';
end if;

-- Resolve the sender side (source coord | target coord/analyst).
if app.is_staff_admin_of(v_ref.source_commission_id) then
  v_sender := v_ref.source_commission_id;
elsif app.is_staff_admin_of(v_ref.target_commission_id)
      or app.referral_target_analyst(p_referral_id, v_uid) then
  v_sender := v_ref.target_commission_id;
else
  raise exception 'apenas coordenadores ou o analista do destino podem enviar mensagens'
    using errcode = 'HC0A0';
end if;

if v_ref.status not in ('sent', 'received', 'accepted', 'in_review', 'awaiting_information') then
  raise exception 'não é possível enviar mensagens neste estado do encaminhamento'
    using errcode = 'HC0A0';
end if;
```

TAKEAWAYS

- In READ COMMITTED (the default), a plain `SELECT` gives you a snapshot that may already be stale by the next statement. `FOR UPDATE` is how you make the check-then-act atomic without escalating the isolation level.
- The lock is held until COMMIT or ROLLBACK — you cannot release it early. Keep the transaction short and always lock rows in a consistent order across your codebase to avoid deadlocks.
- Know the weaker variants: `FOR NO KEY UPDATE` (allows concurrent FK checks), `FOR SHARE`, and `FOR KEY SHARE`. Taking the strongest lock by reflex costs concurrency.
- `FOR UPDATE NOWAIT` errors instead of waiting; `SKIP LOCKED` silently passes over locked rows — the standard building block for a work-queue consumer.
- `select ... into` in plpgsql does not raise when no row matches; it leaves the record NULL. The explicit `if v_ref.id is null` check is required, not defensive padding. (`STRICT` would raise instead.)

20 QUERY DESIGN · SET-BASED THINKING

unnest(...) WITH ORDINALITY turns an array parameter into a set — reorder in one statement

A reorder RPC receives the desired order as a `uuid[]`. Rather than looping in plpgsql with one UPDATE per element, `unnest ... WITH ORDINALITY` materialises the array as a two-column relation of (id, position), which then drives a single set-based `UPDATE ... FROM`. The tenant predicate stays in the same statement, so a foreign id in the array simply matches nothing.

supabase/migrations/20260620000000_baseline.sql:13895-13913

```
CREATE OR REPLACE FUNCTION "public"."reorder_case_outcomes"(
    "p_commission_id" "uuid", "p_ordered_ids" "uuid"[]) RETURNS "void"
    LANGUAGE "plpgsql"
    SET "search_path" TO 'app', 'public', 'pg_catalog'
    AS $$
begin
  perform app.assert_extras_enabled();
  if not (app.is_staff_admin_of(p_commission_id)
          or app.is_org_admin_of_commission(p_commission_id)) then
    raise exception 'sem permissão' using errcode = '42501';
  end if;

  update public.case_outcomes d
  set position = o.ord, updated_at = now()
  from (
    select id, ordinality::integer as ord
    from unnest(p_ordered_ids) with ordinality as t(id, ordinality)
  ) o
  where d.commission_id = p_commission_id and d.id = o.id;
end;
$$;
```

TAKEAWAYS

- `WITH ORDINALITY` may be appended to any set-returning function in the FROM clause and adds a 1-based position column. It is the correct way to preserve input order — array order is otherwise lost the moment you unnest.
- Always alias both columns (`as t(id, ordinality)`); the default naming is unhelpful and version-dependent.
- `UPDATE ... FROM <subquery>` is the Postgres way to do a correlated bulk update. One statement, one plan, one pass — versus a plpgsql loop that pays parse and execute overhead per row.
- Keeping the authorization predicate (`d.commission_id = p_commission_id`) in the same statement means the tenant filter is applied by the same query that writes. A caller passing another tenant's ids updates zero rows rather than being caught by a later check.
- Passing an array to a single RPC also collapses N network round trips into one — usually a larger win than the SQL optimisation itself.

How to keep learning from this codebase

THE CATALOG IS THE TRUTH, NOT THE MIGRATION FILE

- Some migrations here rewrite function bodies at runtime via `pg_get_functiondef()` + `replace()` + `execute`. The file text is therefore a record of *intent*, not of what is installed.
- Query the live catalog instead: `pg_proc` (especially `prosecdef` and `proacl`), `pg_policies`, `pg_policy`, `pg_trigger`, `pg_constraint`, `pg_indexes`.
- Useful psql shortcuts: `\d+ table`, `\df+ schema.*`, `\dp table`, `\sf function_name`.

THREE HABITS THESE PATTERNS SHARE

- **Fail closed.** Every default in this schema — the missing write policy, the `ELSE false` CHECK arm, the `coalesce(current_setting(...), 'off')` — denies rather than permits when something was not thought about.
- **Push the rule as far down as it will go.** A foreign key beats a trigger; a trigger beats application code; application code beats a code-review convention. Several snippets here are the story of a rule being promoted one level down.
- **Say what a statement is for.** Nearly every object carries a `COMMENT ON`. It survives in the catalog, so the next person reads it from the database rather than hunting for the migration.

WORTH EXPLORING NEXT, IN THIS SAME TREE

- `jsonb_build_object` / `jsonb_agg` assembling a whole page payload in one round trip — see `public.session_context()` in `20260905000200_mem_w2_session_context.sql`.
- `LEFT JOIN LATERAL ... ON true` for a correlated per-row lookup — `20260620000000_baseline.sql` around line 10704.
- Catalog-driven refactoring: rewriting ~145 functions and policies programmatically with `pg_get_functiondef`, `pg_policies` and `format('%I')` for injection-safe identifier quoting — `20260709000200_commission_admin_predicate.sql`, lines 125-186.
- Window functions for per-group deduplication — `row_number() over (partition by ...)` in the dashboard aggregates, `20260620000000_baseline.sql` around line 10015.
- The pgTAP suites in `supabase/tests/` — how each of these invariants is actually asserted, including tests that deliberately assume a role and try to break a policy.